

Anchoring Critic-Guided Fine-Tuning of Generative Robot Policies

Anonymous Authors

Abstract— Reinforcement learning can improve robot policies learned from demonstrations, but errors in a learned critic can also destroy their existing skills. We present CAST (Constrained Action-Space Transport), a method for fine-tuning generative robot policies using task-success rewards. CAST learns corrections to a frozen pretrained policy: it adjusts sampled actions using the critic and trains a small network to reproduce them. A restoring term permits small corrections while resisting larger departures from the pretrained behavior. On eight challenging LIBERO-90 tasks, per-task fine-tuning raises mean success from 66.1% to 75.5%, compared with 67.0% for the standard residual actor update. On robomimic square, success rises from 38.2% to 91.2–93.0%. Removing the restoring term causes zero success on all five LIBERO ablation tasks, even with step clipping. The key is to limit accumulated departure from the pretrained policy, not merely the size of each update.

I. Introduction

Learning from demonstrations gives robots a useful starting point, but a policy that usually succeeds may still need practice to become reliable. Reinforcement learning can provide that practice without requiring additional demonstrations, and recent methods apply it to generative policies that produce sequences of robot actions [1], [2]. The challenge is to improve an existing skill while preserving the behavior that already works. We study this problem when feedback is limited to task success, so failed attempts provide little information about how to recover.

A practical approach is residual fine-tuning: freeze the pretrained policy and learn a correction to its actions, guided by a learned critic [3]. The critic predicts the return of an action, and the correction network is trained to increase that prediction. However, critic accuracy is limited by the data available during training. Optimizing its predictions can exploit errors outside well-covered regions of the action space [4], [5]. The policy may then move away from successful pretrained behavior even as its predicted return increases. With success-only rewards, losing that behavior also reduces the successful experience available to correct the critic.

This raises a concrete design question: should fine-tuning limit the size of each update, or the accumulated departure from the pretrained policy? These are different constraints. Many small updates can move a policy far from its starting point. A restoring term tied to a frozen reference instead opposes that accumulated departure. Yet restoring too strongly everywhere can suppress the corrections needed to improve. The desired behavior is to permit local changes while resisting larger ones. Fig. 1 illustrates why this distinction matters.

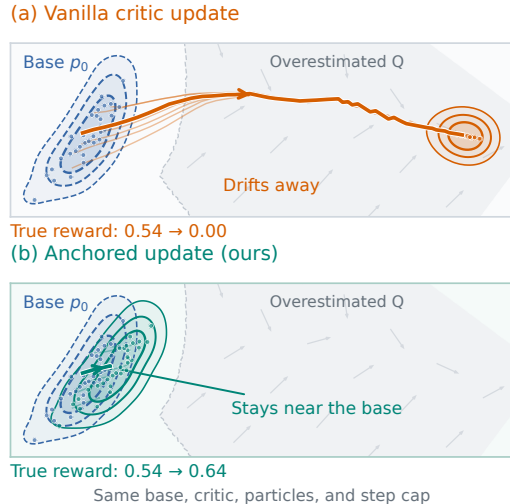


Fig. 1. Same critic, different policy update. Both panels start from the same blue base distribution and 384 paired actions. (a) Vanilla critic ascent moves probability mass into the overestimated region (gray). (b) Anchoring permits improvement near the base. Colored contours enclose 50/80/95% of the estimated mass; paths show representative updates in shared, rotated action coordinates. Scores are particle averages. This single-seed toy uses 25 critic-training samples, 60 updates, and a 0.06 step cap. Radius-0.15 per-particle projection illustrates CAST’s anchor, rather than full policy training.

We present CAST (Constrained Action-Space Transport), a residual fine-tuning method that implements this principle. At each update, it samples actions from the frozen base plus the current correction, moves them along the critic’s action gradient, and trains the correction network to reproduce the adjusted actions. This action-improvement approach builds on prior critic-guided training of generative policies [6], [7]. CAST combines displacement clipping with a restoring field toward the corresponding frozen-base action. A dead zone in this field leaves small corrections unpenalized by that term and pulls back larger departures. The adjusted actions become fixed regression targets, so the actor update requires neither policy likelihoods nor differentiation through the frozen generator.

We evaluate the method on simulated manipulation tasks from LIBERO and robomimic, using matched base policies, critic-training procedures, and interaction budgets for the residual-actor comparison. On eight challenging LIBERO-90 tasks, per-task fine-tuning improves mean success by 9.4 percentage points, compared with 0.9 points for the standard DICE-RL actor. On

robomimic square, CAST reaches 91.2–93.0% success from a 38.2% base, compared with 86.1–90.9% for the standard actor at the reported budget. These gains are conditional: staying near the base can also limit improvement when a task requires substantially different actions.

Our five-task LIBERO ablation isolates the two constraints. Removing all base-restoring terms causes zero success, even with step clipping. Restoration alone avoids complete collapse but is inconsistent across tasks; the full combination improves on every task’s base. A dead-zone penalty in the standard actor’s loss also enables improvement. These results identify restoring geometry as a useful principle shared by both actor updates.

Our contributions are:

- A controlled study distinguishing per-update clipping from fixed-base restoration in critic-guided robot policy fine-tuning.
- CAST, a likelihood-free residual update that combines critic-guided action targets with clipping and base-restoring fields.
- Manipulation experiments identifying the benefits and limits of anchoring, including its transfer to a standard actor’s loss.

II. Related Work

A. Fine-Tuning Generative Robot Policies

Generative policies require specialized reinforcement learning updates because their action likelihoods are often intractable. DPPO optimizes a diffusion policy through its denoising process [1], while ReinFlow introduces stochastic transitions into flow sampling to enable policy-gradient updates [2]. Flow Matching Policy Gradients uses differences in flow-matching losses to construct a policy-update surrogate [8]. These methods exploit the structure of the generator or its training objective. In contrast, CAST needs only samples from a frozen noise-to-action map and a critic. This makes its residual update applicable to one-step generators as well as iterative samplers. Our focus is how to preserve pretrained behavior during critic-guided improvement.

B. Critic-Guided Action Improvement

Using a critic to improve actions before fitting a policy is an established approach. DIPO modifies replay-buffer actions by gradient ascent and trains a diffusion policy on the resulting targets [6]; Q-Score Matching trains a diffusion policy using the critic’s action gradient [7]. Particle-based methods provide another connection: Soft Q-Learning trains an amortized sampler using Stein variational updates [9], and drifting models train one-step generators by fitting displaced samples [10]. Drifting Field Policy uses critic-ranked candidates to guide updates on a drifting backbone [11]. CAST builds on these ideas by constructing fresh action targets for a residual on a frozen base, with displacement clipping and explicit

restoration toward the base. Stored transitions are left unchanged.

An alternative is to use critic values without following their action gradients. AWR and MPO favor actions through advantage- or value-weighted fitting [12], [13], while IDQL combines a diffusion behavior model with critic-based action selection [14]. These approaches motivate our ranking-only and value-weighted field variants. We compare them with the gradient field to distinguish the value of the improvement signal from the effect of anchoring.

C. Residual Learning and Pretrained-Policy Anchoring

Residual learning preserves a base controller while learning corrections. DICE-RL applies this construction to generative policies, combining a critic-driven residual actor with behavior-cloning regularization [3]. It is our closest system-level reference and the source of our controlled actor comparison. DAWN studies the critic cold-start and scaling issues that affect residual RL [15]. More broadly, TD3 and BCQ show how function-approximation and extrapolation errors can undermine critic-guided optimization [4], [5].

Our contribution is a controlled study of the restoring constraint: limiting each displacement, pulling toward a fixed base, and allowing a region of small corrections have different effects. The successful loss-space hinge control is important to this positioning. It shows that useful restoring geometry can be expressed within a conventional actor loss as well as through CAST’s action targets. We therefore study when that geometry preserves and improves pretrained skills, rather than claiming that target regression alone resolves critic error.

III. Problem Statement

A. Setting

We consider online fine-tuning of a generative robot policy pretrained from demonstrations. An observation s includes the inputs used by the policy, and an action $a \in \mathbb{R}^{Hd}$ is a chunk of H controls with d dimensions each. A frozen generator π_0 maps s and noise z to an action. Following residual RL [3], we learn a correction:

$$a_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z), \quad z \sim \mathcal{N}(0, I). \quad (1)$$

An ensemble critic estimates return from replayed experience. Reward is provided on task success. The goal is to increase success through online interaction while retaining useful pretrained behavior. Our controlled comparison varies the actor update while matching the base, critic-training procedure, and interaction budget.

B. The Problem: Accumulated Departure

A critic can assign high values to actions outside its training coverage. Repeatedly following those predictions may move the policy away from actions that actually succeed [5]. A step cap alone does not prevent this: $\|a_{t+1} - a_t\| \leq \delta$ permits distance from the initial action

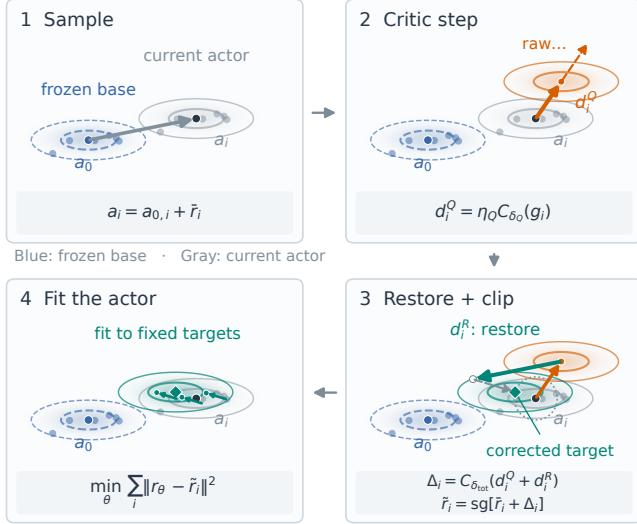


Fig. 2. One coherent CAST update, read clockwise. The blue base and gray current-actor distributions are identical in all four panels. The orange proposal from step 2 carries into step 3; the corrected green targets carry into step 4 for regression. Here $a_{0,i} = \pi_0(s, z_i)$ and $g_i = \nabla_a \bar{Q}_\phi(s, z_i, a_i) / q_{\text{scale}}$. This constructed schematic follows (2)–(6) with illustrative parameters and a shared coordinate scale. The dashed raw request in step 2 continues off-panel. The green cloud denotes fixed targets, not a guaranteed post-update policy.

to grow with the number of updates. Here t indexes optimization steps, not control steps. Fig. 1 compares two updates from the same base distribution under the same critic. Vanilla ascent moves probability mass into an overestimated region; anchoring each particle to its own base action permits improvement nearby.

The design question is how to permit useful corrections without allowing uncontrolled departure from the frozen base. We measure this departure by the residual’s root-mean-square action magnitude, $D_{\text{base}} = \mathbb{E}_{s,z}[\|r_\theta(s, z)\|_2 / \sqrt{Hd}]$, with states sampled from replay. A fixed-base restoring term acts on this departure, whereas step clipping limits only each proposed displacement. We study both empirically: bounded regression targets do not themselves guarantee bounded outputs after fitting a shared neural network.

IV. Method

CAST constrains the actions the actor is asked to learn. We adopt DICE-RL’s frozen-base residual architecture and off-policy training loop [3], replacing its actor objective with regression onto corrected action targets. Fig. 2 follows one update: sample current actions, displace them using the critic, apply restoration and clipping, and fit the residual to the resulting targets. Algorithm 1 places these steps in the online learning loop. All action-space operations use normalized coordinates, with dimension $D = Hd$.

A. Sample and Improve Actions

Step 1: sample (Fig. 2, upper left). For each replay state s , draw K noise samples. Pair each base action

$a_{0,i} = \pi_0(s, z_i)$ with its current residual:

$$\tilde{r}_i = \text{sg}[r_\theta(s, z_i)], \quad a_i = \pi_0(s, z_i) + \tilde{r}_i, \quad (2)$$

where sg denotes stop-gradient. The blue base and gray current-actor distributions in the figure remain fixed during target construction; each correction refers to the base action with the same noise.

Step 2: improve (upper right). Let $\bar{Q}_\phi(s, z, a)$ be the conservative ensemble aggregate, omitting z when the critic does not use it. Normalize its action gradient by the detached scale $q_{\text{scale}} = \max(\epsilon, \mathbb{E}_{\text{batch}}[|\bar{Q}_\phi|])$, computed over the current candidates. The orange displacement is

$$d_i^Q = \eta_Q C_{\delta_Q} \left(\frac{\nabla_a \bar{Q}_\phi(s, z_i, a_i)}{q_{\text{scale}}} \right), \quad (3)$$

where $C_\delta(v) = v \min(1, \delta / \|v\|_2)$, with $C_\delta(0) = 0$. The solid orange arrow is the accepted displacement; the dashed continuation shows the critic’s larger request. Clipping preserves small gradients and rescales large ones before applying η_Q . We differentiate the critic with respect to actions only, holding its parameters fixed for this step.

B. Constrain the Target

Step 3: restore and clip (lower right). A small step can still accumulate into a large departure. We therefore measure each candidate’s distance from its frozen reference in rms units, $m_i = \|\tilde{r}_i\|_2 / \sqrt{D}$, and add the restoring displacement

$$d_i^R = -\eta_0 \tilde{r}_i - \eta_{\text{anc}} \left(1 - \frac{\rho}{\max(m_i, \epsilon)} \right)_+ \tilde{r}_i, \quad (4)$$

where $(x)_+ = \max(x, 0)$. The weak first term always pulls toward the base. The second permits a dead zone of radius ρ and opposes only the excess departure beyond it. The rms definition keeps the radius comparable across chunk dimensions. Unlike a reference to the preceding iterate, the frozen base does not move as training progresses.

The green arrow in Fig. 2 adds d_i^R head-to-tail to the orange proposal. This visual translation represents vector addition: the restoring field is evaluated at $\tilde{r}_i$, not at the displaced action. We then cap the combined displacement and detach the target:

$$\tilde{r}_i = \text{sg} \left[\tilde{r}_i + C_{\delta_{\text{tot}}}(d_i^Q + d_i^R) \right]. \quad (5)$$

The green diamond lies within δ_{tot} of the current action, as indicated by the dotted circle. Thus $\|\tilde{r}_i - \tilde{r}_i\|_2 \leq \delta_{\text{tot}}$. The two controls serve different purposes: restoration opposes accumulated departure from the base, while clipping limits the correction requested in one update. Restoration is soft; it does not project the target into the base-centered dead zone.

Algorithm 1 CAST residual fine-tuning

Require: Frozen base π_0 , demonstrations $\mathcal{D}$, particles K , field and optimization settings

```
1: Initialize  $r_\theta, Q_\phi$ ; set  $\phi^- \leftarrow \phi$ 
2: Seed online replay  $\mathcal{R}$  with base-policy rollouts
3: for each online iteration do
4:   Collect rollouts using the baseline’s warm-up and
   value-guided exploration; append transitions to  $\mathcal{R}$ 
5:   for each gradient step do
6:     Sample a minibatch from  $\mathcal{R}$  and  $\mathcal{D}$ 
7:     Form the multi-sample  $n$ -step TD loss using  $Q_{\phi^-}$ 
8:     if an actor update is due then
9:       Draw  $K$  noises per state; form  $a_i, \bar{r}_i$  via (2)
10:      Form normalized critic steps  $d_i^Q$  via (3)
11:      Evaluate  $d_i^R$  at  $\bar{r}_i$  via (4)
12:       $\tilde{r}_i \leftarrow \text{sg}[\bar{r}_i + C_{\delta_{\text{tot}}}(d_i^Q + d_i^R)]$ 
13:      Update  $\theta$  by regression via (6)
14:    end if
15:    Update  $\phi$  with the critic TD loss
16:    Polyak-update  $\phi^-$  when scheduled
17:  end for
18: end for
19: return  $\pi_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z)$ 
```

C. Fit the Residual

Step 4: regress (lower left). The corrected green cloud from step 3 becomes a set of fixed targets. We fit the residual network with mean-squared error:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_s \left[\frac{1}{KD} \sum_{i=1}^K \|r_\theta(s, z_i) - \tilde{r}_i\|_2^2 \right]. \quad (6)$$

Only this loss updates the actor. The base remains frozen, and the detached targets prevent the actor loss from differentiating through the critic. Freshly constructed targets leave executed replay transitions intact. The method needs no policy likelihood; deployment uses $\pi_0(s, z) + r_\theta(s, z)$.

The constraint bounds the requested target displacement, not the realized change after fitting a shared neural network. Similarly, the dead-zone field has a loss-space counterpart proportional to $(\|r\|_2 - \sqrt{D}\rho)_+^2$, but this correspondence does not make the finite training updates equivalent. Section V compares the restoring field with a hinge penalty and a hard target projection.

V. Experiments

We evaluate the actor update and the constraints that govern its departure from the pretrained policy. Controlled comparisons match the frozen base, critic-training procedure, replay settings, and interaction budget.

A. Training Setup

We alternate online collection with critic and actor updates, using the DICE-RL training framework [3]. The critic retains ensemble temporal-difference learning, n -step returns, and demonstration replay. Online replay is initialized with base-policy rollouts, and minibatches mix online experience with demonstrations. Exploration

uses the baseline’s warm-up and value-guided candidate-selection procedure. Actor updates and target-critic updates follow their configured schedules. These settings are held fixed within each controlled actor comparison.

Actor-update settings. The LIBERO recipe uses $K = 16$, $\eta_0 = 0.05$, $\eta_{\text{anc}} = 1$, $\rho = 0.05$ rms, $\delta_Q = 1$, and $\delta_{\text{tot}} = 0.15$. We compare shared reward steps $\eta_Q \in \{1, 2\}$ across tasks. The robomimic recipe uses $\eta_Q = 0.5$ and $\rho = 0$. Thus the dead-zone setting is benchmark-specific, while the target-construction rule is the same.

B. Constraint and Reward-Field Controls

Constraint factorial. The constraint factorial independently switches step clipping and restoration on or off. Disabling clipping removes both C_{δ_Q} and $C_{\delta_{\text{tot}}}$; disabling restoration sets both η_0 and η_{anc} to zero. Setting $\rho = 0$ instead makes the restoring terms always active, and increasing ρ alone leaves the weak pull intact.

Alternative restoring mechanisms. We compare full CAST with a dead-zone hinge in the backpropagated actor’s loss and a hard projection of the action target. The LIBERO hinge control uses radius 0.05 rms and BC weight 100. The LIBERO projection control caps the residual target at 0.05 rms, with both soft restoring terms and step clips disabled. These comparisons test the implementation of the constraint as well as its shape; related geometry need not produce equally reliable finite training updates.

Reward-field ablations. For reward-field ablations, we replace the action gradient with particle transport [10] toward either the critic’s top- k candidates or candidates weighted by a softmax of standardized Q -values. Clipping, restoration, and regression remain the same, isolating the contribution of the critic’s gradient direction.

References

- [1] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, “Diffusion policy policy optimization,” in International Conference on Learning Representations (ICLR), 2025, arXiv:2409.00588.
- [2] T. Zhang, C. Yu, S. Su, and Y. Wang, “ReinFlow: Fine-tuning flow matching policy with online reinforcement learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2505.22094>
- [3] Z. Sun and S. Song, “From prior to pro: Efficient skill mastery via distribution contractive RL finetuning,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.10263>
- [4] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in International Conference on Machine Learning (ICML), 2018.
- [5] S. Fujimoto, D. Meger, and D. Precup, “Off-policy deep reinforcement learning without exploration,” in International Conference on Machine Learning (ICML), 2019.
- [6] Z. Li, R. Krohn, T. Chen, A. Ajay, P. Agrawal, and G. Chalvatzaki, “Learning multimodal behaviors from scratch with diffusion policy gradient,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.00681>
- [7] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a diffusion model policy from rewards via Q-score matching,” 2025. [Online]. Available: <https://arxiv.org/abs/2312.11752>
- [8] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, “Flow matching policy gradients,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.21053>

- [9] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in International Conference on Machine Learning (ICML), 2017.
- [10] M. Deng, H. Li, T. Li, Y. Du, and K. He, "Generative modeling via drifting," 2026. [Online]. Available: <https://arxiv.org/abs/2602.04770>
- [11] J. Koo, M. Park, J. Choi, Y. Min, and M. Sung, "Drifting field policy: A one-step generative policy via wasserstein gradient flow," 2026. [Online]. Available: <https://arxiv.org/abs/2605.07727>
- [12] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, "Advantage-weighted regression: Simple and scalable off-policy reinforcement learning," arXiv preprint arXiv:1910.00177, 2019.
- [13] A. Abdolmaleki, J. T. Springenberg, Y. Tassa, R. Munos, N. Heess, and M. Riedmiller, "Maximum a posteriori policy optimisation," in International Conference on Learning Representations (ICLR), 2018.
- [14] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, "IDQL: Implicit Q-learning as an actor-critic method with diffusion policies," arXiv preprint arXiv:2304.10573, 2023.
- [15] G. Ma, L. Li, H. Wang, Z. Liu, P.-L. Bacon, and D. Tao, "What makes value learning efficient in residual reinforcement learning?" 2026. [Online]. Available: <https://arxiv.org/abs/2602.10539>